

# Key-Value Store v1

Multithread, sezioni critiche, safety e liveness

Architetture dei Sistemi Distribuiti

Lezione 10

Nuovo scenario:

- stesso servizio key-value;
- piu' client simultanei;
- un thread per connessione;
- stato interno condiviso.

Il problema centrale non e' piu' la sintassi del protocollo, ma la correttezza degli interleaving.

# Perche' questa tappa

La concorrenza rende visibili nuovi rischi:

- race condition;
- lost update;
- osservazione di stato incoerente;
- lock troppo larghi che peggiorano liveness.

# Nuovo comando utile: INCR

Introduciamo:

- INCR <key>

Questo comando rende esplicito un percorso read-modify-write:

- 1 leggere il valore attuale;
- 2 calcolare il nuovo valore;
- 3 riscrivere il risultato.

Tre famiglie:

- read-only: GET, EXISTS, KEYS;
- write semplici: SET, DELETE;
- read-modify-write: INCR.

Il percorso piu' delicato e' INCR, perche' legge, decide e poi aggiorna stato condiviso.

# Interleaving unsafe

Stato iniziale:

```
counter = 10
```

Possibile interleaving:

```
T1: read counter -> 10  
T2: read counter -> 10  
T1: write counter <- 11  
T2: write counter <- 11
```

Risultato: un incremento si perde.

## Percorso unsafe nel codice

```
current = self._data.get(key, "0")
numeric_value = int(current)
time.sleep(INCR_DELAY_SECONDS)
numeric_value += 1
self._data[key] = str(numeric_value)
```

Senza lock, due thread possono attraversare insieme questo percorso e perdere un aggiornamento.

```
with self._lock:  
    current = self._data.get(key, "0")  
    numeric_value = int(current)  
    time.sleep(INCR_DELAY_SECONDS)  
    numeric_value += 1  
    self._data[key] = str(numeric_value)
```

La sezione critica deve coprire l'intera sequenza read-modify-write.



## Safety

Non accade mai qualcosa di scorretto.

Nel nostro contesto:

- niente lost update su INCR;
- stato condiviso sempre coerente;
- operazioni osservano una disciplina di serializzazione chiara.

## Liveness

Il sistema continua a fare progresso.

Nel nostro contesto:

- i thread non restano bloccati per sempre;
- il server continua a servire richieste;
- la sezione critica non deve inglobare I/O di rete.

Una soluzione con lock puo' migliorare safety ma peggiorare:

- throughput;
- tempi di attesa;
- rischio di starvation;
- evolvibilita' del server.

Regola pratica della tappa:

- proteggere tutto cio' che tocca stato condiviso;
- non trattenere il lock durante I/O di rete;
- non introdurre lock annidati senza necessita'.

Una richiesta INCR puo' essere vista come automa:

- Idle -> ReadCurrent -> ComputeNext -> WriteBack -> Reply

Nella versione safe:

- Idle -> AcquireLock -> ReadCurrent -> ComputeNext -> WriteBack -> ReleaseLock -> Reply

La domanda tecnica e': quale sottosequenza deve essere atomica?

Durante il laboratorio chiedetevi:

- quale parte della transizione deve essere atomica;
- quale stato e' locale al thread e quale e' condiviso;
- quali interleaving violano safety;
- quali interleaving restano ammessi ma sono solo piu' lenti.

In un software concorrente il contratto non dipende solo dall'API.

Dipende anche da:

- quali interleaving sono ammessi;
- quali porzioni di codice sono atomiche;
- quali proprietà di safety e liveness vengono difese.

Lavoro richiesto:

- ① analizzare la versione unsafe;
- ② riprodurre una race con `stress_incr.py`;
- ③ introdurre lock e sezioni critiche;
- ④ discutere l'effetto sui percorsi di safety e liveness.